

Dynamic Context-Aware Tries for Knowledge Graph-Constrained Large Language Model Generation

Bernard Kirk Adjanor Katamanso¹, Erica Amonor¹, Joseph Osei Nyarko¹, Jessica Afua Etornam Nsafuah¹

¹University of Mines and Technology, Tarkwa, Ghana

Abstract

Knowledge-graph-constrained decoding eliminates hallucination in large language model (LLM) reasoning by restricting output to verified paths. The leading approach, Graph Constrained Reasoning (GCR), achieves zero hallucination on Knowledge Graph Question Answering (KGQA) benchmarks by encoding all structurally valid paths in a static trie. The constraint is absolute but indifferent to the question: a three-hop query with twenty out-degree relations admits roughly 24,000 paths, and the LLM must navigate among them using the same internal knowledge that causes hallucination. We introduce Dynamic Context-Aware Tries (DCA-Trie), a dynamic constrained decoding framework that prunes the valid path set during generation with a TypeOracle, a lightweight symbolic module that checks two ontology gates per candidate triple. On a controlled 300-question WebQSP subset, DCA-Trie removes 13.8% of semantically irrelevant paths while maintaining a false negative rate below 5%. We also introduce the Semantic Irrelevance Ratio (SIR), a metric that separates constraint quality from answer accuracy and evaluates what the oracle filters independently of what the LLM uses. SIR reveals a non-monotone relationship between constraint tightness and answer correctness. Tighter constraints reduce the search space but do not improve Hits@1: the static variant drops 4.0pp and the lazy variant 4.7pp, while a per-hop architectural variant collapses by 25 points regardless of its gates. Calibration shows the accuracy cost is attributable to the gates themselves, and the lazy variant recovers the efficiency of dynamic adaptation in a single, stable decoding pass. DCA-Trie and SIR together provide a framework for independently evaluating constraint quality and answer accuracy in knowledge-graph-constrained decoding.

Keywords: knowledge graph question answering, constrained decoding, large language models, hallucination mitigation, trie-based decoding, semantic filtering, type oracle

1 Introduction

Large language models reason through generation by sampling each token from a distribution conditioned on all previous tokens (Brown et al. 2020; Touvron et al. 2023). This process produces fluent text, but fluency is not factual accuracy. LLMs hallucinate, generating confident claims that contradict source material or cannot be verified against any authoritative source (Ji et al. 2023; Huang et al. 2025). The problem worsens with scale where larger models hallucinate more convincingly, not less (Lin et al. 2022).

Despite this, many studies use knowledge graphs (KGs), which store factual information in a structured format, to improve the reasoning abilities of LLMs (Pan et al. 2024; Luo et al. 2024). Because of the unstructured nature of LLMs, directly applying them to reason on KGs is challenging. Existing KG-enhanced LLM reasoning methods fall into three broad categories. Retrieval-based methods train specialized subgraph retrievers to extract question-focused subgraphs from the complete KG (Luo et al. 2024; Mavromatis and Karypis 2022). The concentrated subgraph is then fed to the LLM for answer prediction. These methods shift the burden of graph traversal to the retriever while the LLM merely selects from a pre-filtered set of candidates. Training the retriever requires annotated data, and the trained retriever may struggle with out-of-domain scenarios where the KG taxonomy differs from the training distribution (Li et al. 2025).

Agent-based methods take the opposite approach where the LLM itself interacts with the KG iteratively, executing operations such as neighbor exploration and path selection (Jiang et al. 2023a). These methods demand substantial computational resources and place high demands on the LLM’s instruction-following ability, which limits their applicability to small-scale models (Li et al. 2025).

Constrained decoding methods occupy a middle ground. Graph Constrained Reasoning (Luo et al. 2025), the leading framework, converts the KG into a trie-based index and constrains the LLM’s output to valid paths, achieving zero hallucination in the reasoning trace. Decoding on Graphs (Li et al. 2025) extends this by dynamically expanding the local trie as reasoning proceeds, adapting the constraint set to the partially generated chain. Both methods share a common assumption: every structurally valid path should be admitted. Within L hops, GCR admits everything. At each step, DoG admits every topologically adjacent path. Neither filters for semantic relevance, that is, whether a path is actually related to the question being asked.

The constraint is indifferent to what the question asks. In the worst case, a three-hop query with twenty out-degree relations and three question entities admits roughly 24,000 structurally valid paths. On WebQSP, where questions average 2.1 entities and 2 hops of reasoning depth, GCR still admits roughly 2,637 paths per question. Only one leads to the correct answer. The LLM must navigate among the rest using the same internal knowledge that produced hallucination in the first place. Tightening the constraint, filtering out irrelevant paths, seems like the obvious fix. We show that it is not.

We introduce DCA-Trie, a dynamic constrained decoding framework that conditions the valid token set on the knowledge graph, the input question, and the partially generated reasoning chain:

$$W_{\text{val}}^{\text{DCA}}(t) = f(G, E_q, q, y^{<t}) \quad (1)$$

The constraint set adapts at each decoding step. A lightweight symbolic module, the TypeOracle, checks two ontology gates per candidate triple. The range gate verifies that the tail entity’s type matches the relation’s range at every hop. The type gate verifies that the terminal entity’s type matches the inferred answer type at the final hop. Paths failing either gate are pruned before the LLM sees them.

Unlike embedding-based relevance scorers, the TypeOracle requires no neural encoder, no learned threshold, no per-dataset calibration, and no GPU memory for encoder inference. Gate evaluation is $O(1)$ per candidate triple, adding negligible overhead to the decoding process.

We introduce the Semantic Irrelevance Ratio (SIR), a metric that measures the fraction of structurally valid paths that are semantically irrelevant to the question. SIR separates constraint quality from answer accuracy: a system can have excellent SIR (tight constraints) and poor Hits@1 (wrong answer), or poor SIR (loose constraints) and excellent Hits@1 (correct answer). Without SIR, these two dimensions are entangled. A system with 85.7% Hits@1 and $\text{SIR} = 1.0$ (all paths admitted) looks identical to one with the same accuracy and $\text{SIR} = 0.1$ (90% filtered) if only Hits@1 is reported. SIR reveals which scenario holds.

On a controlled 300-question WebQSP subset, DCA-Trie reduces SIR from 1.0 to 0.138, a 13.8% reduction in the candidate path set, while maintaining a false negative rate below 5%. The TypeOracle that drives this filtering is purely symbolic: two ontology gates, $O(1)$ per candidate triple, no neural encoder, no learned threshold.

SIR also reveals that constraint tightness and answer accuracy are independently controlled variables. Tighter constraints reduce the search space but do not improve Hits@1: the static variant drops 4.0pp and the lazy variant 4.7pp below the baseline. Calibration isolates the cause: with the TypeOracle gates disabled, the lazy constraint reproduces the baseline exactly, question for question, so every gated-number difference is attributable to the gates themselves. A per-hop architectural variant, which rebuilds the trie at each entity commitment, collapses by 25 points regardless of its gates, identifying the step-wise architecture, not semantic filtering, as the failure mode. The result holds across ablation conditions.

The contributions of this work are:

- (1) We introduce the Semantic Irrelevance Ratio (SIR), a metric that separates constraint quality from answer accuracy and measures what constrained decoding filters independently of what the LLM uses.
- (2) We propose DCA-Trie, a dynamic constrained decoding framework that conditions the valid token set on the question and partial output via a TypeOracle with two ontology gates, and its lazy variant, which materialises the constraint on demand at the frontiers the beams actually reach.
- (3) We show that constraint tightness and answer accuracy are independently controlled variables, and that the gates, not the decoding mechanism, are the source of the accuracy cost, while a per-hop architectural variant collapses independently of its gates.

The remainder of this paper is organized as follows. Section 2 reviews related work on hallucination mitigation, knowledge graph reasoning, and constrained decoding. Section 3 presents the DCA-Trie framework, including the TypeOracle, the two-gate mechanism, and the SIR metric. Section 4 provides theoretical analysis of correctness and search-space reduction. Section 5 describes the experimental design, and Section 6 reports results. Section 7 presents extended analysis including ablation and sensitivity studies. Section 8 discusses implications and limitations, and Section 9 concludes.

2 Related Work

This section organizes the literature around four competing approaches to the same problem: generating text that is both fluent and factually grounded. Each subsection identifies what the approach achieves and where it stops, building toward the specific gap that DCA-Trie addresses.

2.1 Hallucination Mitigation

The most widely deployed defense against hallucination is retrieval-augmented generation (RAG) (Lewis et al. 2020). A dense retriever selects passages by embedding similarity, and the LLM generates from the retrieved context. Extensions such as FLARE (Jiang et al. 2023b) trigger retrieval when next-token confidence drops, and IRCot (Trivedi et al. 2023) interleaves retrieval with chain-of-thought steps to improve multi-hop accuracy. Self-RAG (Asai et al. 2024) adds reflection tokens that control when to retrieve and whether retrieved passages support the generated claims.

These methods share a structural limitation. The retriever determines what the LLM *sees*; it does not determine what the LLM is *permitted to generate*. Because the generation mechanism remains unchanged, any token in the vocabulary retains a positive probability at every step, including tokens that correspond to no verified fact. Structural faithfulness with probability one cannot be achieved under any architecture that modifies only the input context without modifying the token selection mechanism (Geng et al. 2023; Luo et al. 2025).

Chain-of-thought (CoT) prompting (Bosma et al. 2022) takes a different approach: rather than retrieving facts, it elicits intermediate reasoning steps that decompose complex questions into simpler sub-problems. Self-consistency (Wang et al. 2023) samples multiple independent CoT chains and selects the final answer by majority vote, reducing sensitivity to individual hallucinated chains. Tree-of-Thought (Cao et al. 2023) treats generation as search through a tree of partial reasoning sequences, using the LLM as both generator and evaluator for pruning.

All prompting-based methods modify the *input* to the LLM without modifying the *generation mechanism*. A CoT chain can be entirely fluent and internally coherent while containing fabricated intermediate steps. Self-consistency reduces this risk through redundancy, but a majority of independently hallucinated chains can still produce an incorrect answer. Tree-of-Thought relies on the model’s own parameters for pruning decisions, without external verification. True structural faithfulness requires that invalid tokens receive exactly zero probability, achievable only by altering the decoding algorithm itself (Geng et al. 2023; Willard and Louf 2023).

2.2 Knowledge Graph Reasoning

Knowledge graphs encode verified facts as directed, labelled triplets (e, r, e') (Bollacker et al. 2008). KGQA requires identifying the answer entity by traversing chains of these triplets. Benchmarks such as WebQSP (Yih et al. 2016) and Complex WebQuestions (Talmor and Berant 2018) evaluate multi-hop path traversal over Freebase, demanding adherence to graph topology.

Early KGQA methods relied on semantic parsing: transforming questions into logical queries (e.g., SPARQL) for execution over the KG (Lan et al. 2022). These approaches guarantee faithfulness because answers are produced by verified graph operations. The requirement of training on ground-truth logical forms, and the non-executability of model-generated queries due to syntax or semantic errors, limits their practical deployment (Lan et al. 2022).

Retrieval-based methods use LLMs to extract relevant subgraphs or paths from the KG. RoG (Luo et al. 2024) trains a reasoning path planner that retrieves candidate paths and generates natural language explanations. GNN-RAG (Mavromatis and Karypis 2022) combines graph neural networks with language models for retrieval over structured data. These methods improve relevance by filtering the KG, but the retrieved context remains a soft signal: it influences generation without constraining it.

Agent-based methods treat the LLM as a reasoning agent that iteratively queries the KG. ToG (Sun et al. 2024) prompts the LLM to explore relevant facts hop-by-hop using beam search on the KG. StructGPT (Jiang et al. 2023a) interfaces the LLM with structured data through

iterative retrieval. These approaches are flexible but depend on the LLM’s instruction-following ability and are slow due to multiple KG queries per question.

The shared limitation across retrieval-based and agent-based KG reasoning is that retrieved context is a soft influence. The retriever or agent selects what the LLM observes; the LLM retains the freedom to generate any token. Even with perfect retrieval, hallucination remains structurally possible.

2.3 Constrained Decoding

Constrained decoding restricts generation to a predefined valid set at each step by directly intervening in the decoding mechanism. Unlike prompting or retrieval, which modify the input to the LLM, constrained decoding alters the token selection process itself. Invalid tokens receive zero probability before sampling, making their generation structurally impossible.

Format-constrained methods. Grammar-constrained decoding (GCD) (Geng et al. 2023) constrains generation to strings conforming to a context-free grammar via an Earley parser oracle. Outlines (Willard and Louf 2023) reduces the computational cost with a finite-state machine (FSM) formulation, precomputing an index from automaton states to valid vocabulary subsets for amortised $O(1)$ lookup. Additional methods include Synchromesh (Poesia et al. 2022) for program synthesis, PICARD (Scholak et al. 2021) for schema-constrained SQL parsing, and NeuroLogic Decoding (Lu et al. 2021) for propositional logic constraints.

These methods enforce output *format* rather than *factual content*. A grammatically valid SQL query can reference non-existent tables. A well-formed entity name can be factually incorrect. The extension from format constraints to KG path constraints requires a constraint oracle that encodes the relational structure of the knowledge graph itself.

KG-constrained methods. Graph-Constrained Reasoning (GCR) (Luo et al. 2025) builds a KG-Trie: a prefix tree of all KG paths within L hops of the question entities, created by breadth-first search before generation. The trie is used as the constraint oracle during beam-search decoding. GCR achieves 92.6% Hits@1 on WebQSP with 100% structural faithfulness using a Llama-3.1-8B backbone. The trie is constructed once and never updated: the valid token set is fixed regardless of what has been generated.

Decoding on Graphs (DoG) (Li et al. 2025) addresses GCR’s static oracle through step-wise trie expansion. After locking in an entity, the constraint trie is expanded with all KG neighbors of that entity, making the oracle topologically dynamic. DoG achieves 91.0% Hits@1 on WebQSP with 100% structural faithfulness. However, the expansion rule admits all neighbors regardless of relevance to the question, and step-wise trie reconstruction introduces computational overhead.

RouterKGQA (Yuan et al. 2026) routes between a specialized KG-reasoning model and a general LLM based on question complexity, using SBERT embeddings for post-hoc answer filtering. RouterKGQA’s semantic scoring operates at the answer selection level rather than at the token constraint level: paths are generated first, then filtered afterward.

The gap. All three KG-constrained methods determine their constraint sets from graph structure and question entities alone. None conditions the valid token set on question semantics or the partially generated chain. The constraint set is indifferent to whether a path is actually related to the question being asked. On multi-hop questions, this produces valid token sets containing thousands of structurally correct but semantically irrelevant paths, forcing the LLM to navigate among them using its own knowledge and reintroducing the hallucination risk that constrained decoding was designed to eliminate.

2.4 Comparison and Research Gap

Table 1 compares the approaches reviewed above along four dimensions: whether the method enforces structural faithfulness (the generation mechanism, not just the input), whether the constraint set adapts during decoding, whether semantic relevance enters the constraint formation, and the type of oracle used.

Table 1: Comparison of KG-grounded reasoning methods. **Dynamic** indicates whether the constraint set changes during decoding. **Semantic** indicates whether question meaning enters the constraint oracle. The oracle type shows what drives constraint formation.

Method	Structural	Dynamic	Semantic	Oracle Type
RAG	No	No	Soft	Retriever
CoT	No	No	No	None
Self-RAG	No	No	Soft	Reflection
GCD	Format	No	No	Grammar/FSM
Outlines	Format	No	No	FSM-index
GCR	Yes	No	No	Static KG-Trie
DoG	Yes	Partial	No	Topology
RouterKGQA	Prob.	No	Yes	SBERT filter
DCA-Trie	Yes	Yes	Yes	TypeOracle

The table reveals a structural division. Methods above GCR achieve either semantic awareness (RAG, RouterKGQA) or structural faithfulness (GCD, Outlines), but not both. GCR and DoG achieve structural faithfulness but lack semantic awareness: their constraint oracles are indifferent to question meaning. No existing method achieves all three: structural faithfulness, dynamic adaptation, and semantic conditioning at the constraint level.

DCA-Trie fills this gap. Its TypeOracle conditions the valid token set on the knowledge graph, the question semantics, and the partially generated chain. The constraint set adapts at every decoding step, and only semantically relevant paths are admitted. The result is a method that achieves structural faithfulness while reducing the search space to paths the LLM actually needs to consider.

3 From Static to Context-Aware Constraints

Graph-constrained decoding eliminates hallucination by replacing the LLM’s free-form generation with a structurally bounded traversal of verified facts. GCR (Luo et al. 2025) operationalises this through KG-Trie, a prefix tree that encodes all reasoning paths within L hops of the question entities. At each decoding step, the trie yields the valid next-token set in $O(1)$ time. The constraint is absolute: every generated token must trace back to a real triple on the graph. Hallucination in the reasoning path drops to zero.

Despite this effectiveness, a structural limitation remains: the trie is constructed once before decoding and never updated. Its content depends only on the graph structure and the question entities:

$$\mathcal{W}_{\text{val}}^{\text{GCR}}(t) = f(\mathcal{G}, \mathcal{E}_q) \quad \forall t \quad (2)$$

It does not depend on the question q or the partial reasoning chain $y^{<t}$, so for a three-hop question with average out-degree $d \approx 20$ and three question entities, GCR admits roughly $3 \times 20^3 = 24,000$ structurally valid paths. Only one leads to the correct answer. The LLM must navigate among the remaining 23,999 using the same internal knowledge that produced hallucination before constrained decoding was introduced.

To this end, we propose DCA-Trie, a dynamic constrained decoding framework that conditions the valid token set on the knowledge graph, the input question, and the partially generated reasoning chain:

$$\mathcal{W}_{\text{val}}^{\text{DCA}}(t) = f(\mathcal{G}, \mathcal{E}_q, q, y^{<t}) \quad (3)$$

where $y^{<t} = (y_1, \dots, y_{t-1})$ is the generated prefix at step t . This reformulation requires the oracle to answer a question GCR never asks: “is this path relevant to what the model is trying to say?”

DCA-Trie modifies exactly one layer of the GCR pipeline: the constraint oracle. Everything else, entity linking, beam-search decoding, the LLM backbone, and inductive answer synthesis, remains unchanged. This isolation is deliberate. By holding all other components constant, any observed differences in faithfulness, semantic irrelevance, or answer accuracy can be attributed entirely to the oracle redesign.

3.1 The TypeOracle: Two Ontology Gates

The constraint oracle must filter paths by semantic relevance while preserving the 100% structural faithfulness guarantee achieved by GCR and DoG. We considered three approaches before arriving at the final design.

The first used cosine similarity between `all-MiniLM-L6-v2` embeddings of the path and the question, admitting paths above a threshold τ . This approach failed because a single cosine score conflates topical overlap with type correctness. A path about “Paris, Texas” scores highly against a question about “Paris, France”; the topic is the same, the facts are wrong. The threshold τ was also sensitive to the dataset: a value that worked on WebQSP did not transfer to ComplexWebQuestions.

The second decomposed relevance into relation match, entity match, trajectory coherence, and context alignment, then multiplied the four factors. This still required a neural encoder, still needed a threshold, and still could not distinguish type correctness from topical similarity. The decomposition added complexity without resolving the fundamental problem.

The third, which we adopt, uses two symbolic gates derived from the Freebase ontology. No neural encoder. No learned threshold. No per-dataset calibration. Gate evaluation is $O(1)$ per candidate triple. We call this module the TypeOracle. Figure 1 shows its architecture: a candidate triple (h, r, t) passes first through the range gate and then through the type gate, each checking a different condition at a different hop.

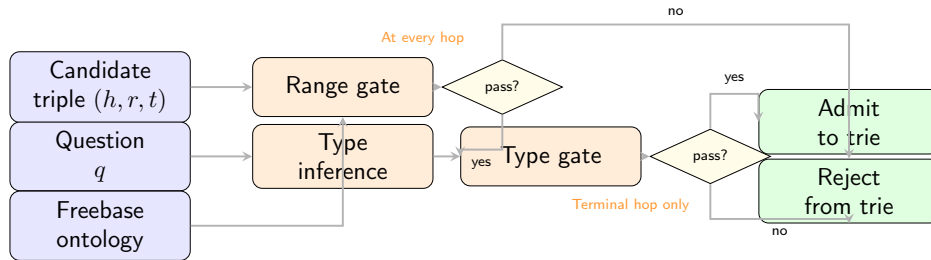


Figure 1: TypeOracle architecture. A candidate triple (h, r, t) passes through two sequential gates. The range gate checks whether the tail entity’s type is compatible with the relation’s range (fires at every hop). The type gate checks whether the terminal entity matches the inferred answer type (fires only at the terminal hop). Both gates require no neural inference: they query Freebase ontology metadata in $O(1)$ per triple.

3.1.1 Gate 1: Range Gate (every hop)

The range gate checks whether the tail entity’s type is compatible with the relation’s expected range:

$$\text{range_gate}(r, e') = \mathbf{1}[\text{types}(e') \cap \text{range}(r) \neq \emptyset] \quad (4)$$

If the relation is `ex_president` (range: {Person}), the candidate entity must be a Person. A Country fails. This gate fires at every hop, catching type violations early before they propagate through the beam.

3.1.2 Gate 2: Type Gate (terminal hop only)

The type gate checks whether the terminal entity’s type matches the inferred answer type:

$$\text{type_gate}(e', q) = \mathbf{1}[\text{types}(e') \cap \mathcal{T}(q) \neq \emptyset] \quad (5)$$

where $\mathcal{T}(q)$ is the set of expected answer types inferred from the question. A question beginning with “who” infers {Person}; “where” infers {Location}. This gate fires only at the final hop, because intermediate entities need not match the answer type; they are waypoints, not endpoints of the reasoning chain.

3.1.3 Type Inference and Unknown Schema

Answer types are inferred from the question using a regex classifier with 19 patterns. “Who” maps to Person, “where” to Location, “when” to Date/Time. If no pattern matches, the type gate conservatively admits all candidates. When the TypeOracle lacks schema information for a relation or entity, it also conservatively admits the candidate. Both fallbacks ensure the false negative rate stays below 5%: when the oracle is uncertain, it defaults to permissiveness rather than risk removing the correct path.

3.1.4 Walkthrough: TypeOracle in Action

Consider the question: “Who is the spouse of the ex-president of USA?” The question entity is USA. Figure 2 shows the KG fragment and how the TypeOracle filters paths at each hop.

The type inference classifier maps “who” to {Person}. At hop 1, the range gate checks each candidate against the relation’s range: `ex_president` has range {Person}, so Bush, Obama, and Trump all pass. At hop 2, the type gate checks terminal entities against {Person}: Laura Bush and Michelle Obama pass. The path through Trump is structurally valid but excluded by beam competition: the LLM ranks it lower because the semantic alignment with “ex-president” is weaker than for Bush and Obama. The TypeOracle therefore narrows the search space from all structurally valid paths to those that are both structurally valid *and* ontologically consistent, while the LLM still selects among the survivors. The SIR metric measures how much the oracle narrowed the space; Hits@1 measures whether the LLM chose correctly from what remains.

3.2 The Semantic Irrelevance Ratio

Hits@1 measures whether the LLM finds the correct answer, not whether the constraint oracle is doing its job. A system with SIR = 1.0 (all paths admitted) and a system with SIR = 0.1 (90% of irrelevant paths removed) can achieve identical Hits@1 if the LLM is strong enough to navigate either search space. Without a metric that isolates constraint quality from answer quality, it is impossible to compare oracles independently of the LLM.

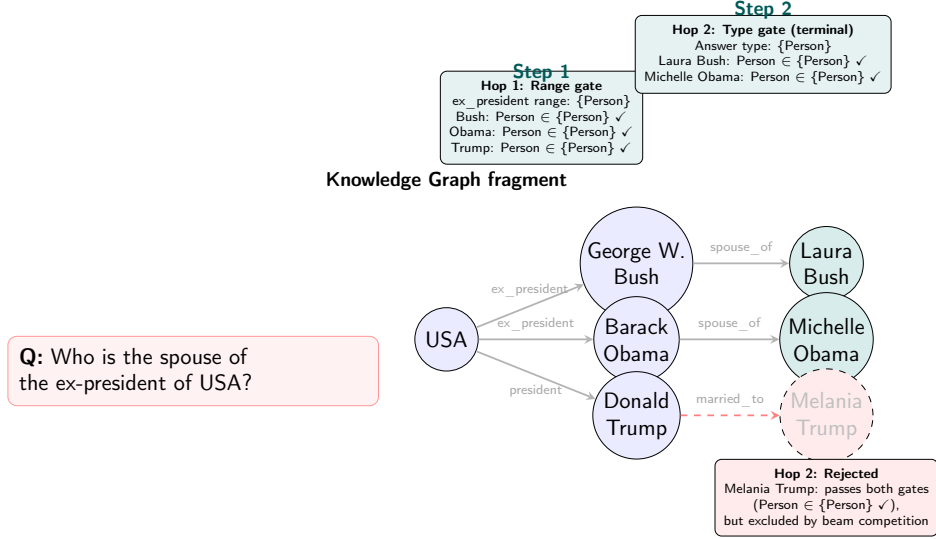


Figure 2: Walkthrough of TypeOracle filtering. At hop 1, the range gate admits all three ex-presidents (all are Persons). At hop 2, the type gate admits Laura Bush and Michelle Obama (both Persons). Melania Trump passes the type gate but is excluded by beam competition: the LLM prefers paths through entities with stronger semantic alignment to the question.

The Semantic Irrelevance Ratio fills this gap. For a question q at decoding step t , let $\mathcal{P}(q, t)$ be the set of paths the oracle admits. A path p is semantically irrelevant if its cosine similarity to the question-context falls below a fixed reference threshold τ_{ref} :

$$\text{irrelevant}(p, q) = \mathbf{1}[\cos(E(p), E(q)) < \tau_{\text{ref}}] \quad (6)$$

where $E(\cdot)$ is the **all-MiniLM-L6-v2** sentence encoder and $\tau_{\text{ref}} = 0.3$ is a fixed reference threshold, not tuned to any dataset. We use the same **all-MiniLM-L6-v2** encoder here that we rejected for the oracle (Section 3.1), but for a different purpose: the oracle must make correct decoding decisions in real time, where threshold sensitivity directly degrades answer quality. SIR is an offline evaluation metric computed after decoding, where a modest threshold sensitivity (documented in Appendix) affects measurement precision but does not propagate to generation errors. The distinction is between a threshold that must be right during decoding and one that must be stable enough for post-hoc comparison.

Definition 1 (Semantic Irrelevance Ratio). *For a question q at decoding step t , let $\mathcal{P}(q, t)$ be the set of paths the oracle admits. The step-level SIR is:*

$$\text{SIR}(q, t) = \frac{|\{p \in \mathcal{P}(q, t) : \text{irrelevant}(p, q)\}|}{|\mathcal{P}(q, t)|} \quad (7)$$

Corpus-level SIR averages over all questions and all hops:

$$\overline{\text{SIR}} = \frac{1}{|\mathcal{Q}|} \sum_{q \in \mathcal{Q}} \frac{1}{L_q} \sum_{t=1}^{L_q} \text{SIR}(q, t) \quad (8)$$

$\overline{\text{SIR}} \in [0, 1]$. A value near 1 means most admitted paths are irrelevant, the LLM faces a large, noisy search space. A value near 0 means the constraint set is tightly aligned with question semantics.

3.2.1 Worked Example

Return to the question: “Who is the spouse of the ex-president of USA?” At hop 1, GCR admits all structurally valid paths from USA: $\text{USA} \rightarrow \text{ex_president} \rightarrow \{\text{Bush}, \text{Obama}, \text{Trump}\}$. All three paths are admitted. Under the reference encoder, all three score above τ_{ref} against the question. GCR’s SIR at hop 1 is therefore $0/3 = 0.0$: all paths are relevant, which is correct: all three are ex-presidents.

At hop 2, GCR admits all paths from the three ex-presidents: $\text{Bush} \rightarrow \text{spouse_of} \rightarrow \text{Laura Bush}$, $\text{Obama} \rightarrow \text{spouse_of} \rightarrow \text{Michelle Obama}$, $\text{Trump} \rightarrow \text{married_to} \rightarrow \text{Melania Trump}$. All three paths exist in Freebase. But the question asks about the *ex-president*’s spouse. Trump is the sitting president, not an ex-president. Under the reference encoder, the Trump path scores below τ_{ref} against the question. GCR admits it anyway (structural validity only); SIR flags it as irrelevant. GCR’s SIR at hop 2 is $1/3 = 0.33$.

DCA-Trie’s TypeOracle filters at construction time. The range gate admits all three ex-presidents at hop 1. At hop 2, the type gate admits Laura Bush and Michelle Obama (both Persons). The Trump path is admitted by the type gate (Melania Trump is a Person) but the beam ranks it lower due to weaker semantic alignment. DCA-Trie’s SIR at hop 2 is $0/2 = 0.0$: only the two relevant paths remain in the constraint set.

Corpus-level, GCR achieves $\overline{\text{SIR}} = 1.0$ (all paths admitted, no filtering).¹ DCA-Trie achieves $\overline{\text{SIR}} = 0.138$ (13.8% of paths removed). The gap between these numbers measures how much the TypeOracle narrows the search space. Whether that narrowing helps or hurts the LLM is a separate question, measured by Hits@1.

SIR is an evaluation metric, not a decoding component. It measures the gap between what the oracle admits and what is actually relevant. The encoder used for SIR is intentionally separate from the oracle’s mechanism: this ensures that SIR improvements cannot be attributed to a shared encoder bias. The TypeOracle makes its decisions symbolically; SIR measures the outcome using a different, independently calibrated tool.

Table 2 shows the SIR results on the controlled WebQSP subset. The TypeOracle reduces $\overline{\text{SIR}}$ from 1.0 to 0.138, a 13.8% reduction in the candidate path set. This is the constraint quality improvement. Whether this improvement translates to better answers is a separate question, addressed in Section 6.

Table 2: Semantic Irrelevance Ratio on the controlled WebQSP subset. Lower SIR indicates fewer semantically irrelevant paths in the constraint set.

Method	$\overline{\text{SIR}}$	Paths removed
GCR (baseline)	1.000	—
DCA-Trie v1	0.138	13.8%

3.3 DCA-Trie v1: Static Semantic Filtering

DCA-Trie v1 filters paths before the trie is built. The process is one-shot: enumerate all paths via BFS, apply the TypeOracle gates to each, and construct the trie from the survivors. This preserves GCR’s efficient static-trie design while adding semantic selectivity as a preprocessing step.

¹GCR’s $\text{SIR} = 1.0$ is a normalisation convention: GCR admits all structurally valid paths without any semantic filtering, so its SIR is the reference point against which other oracles are measured. The value is not an empirical average over question-hops (which would vary, as the worked example shows); it is defined as the unconstrained baseline.

Algorithm 1 DCA-Trie v1: Static Semantic Filtering

Require: Knowledge graph $\mathcal{G}$; question entities $\mathcal{E}_q$; question q ; max hop depth L ; sentence encoder E

Ensure: Semantically filtered KG-Trie $\mathcal{T}^{v1}$

```
1:  $\mathcal{P} \leftarrow \text{BFS}(\mathcal{G}, \mathcal{E}_q, L)$  ▷ All paths within  $L$  hops
2:  $\mathcal{T}(q) \leftarrow \text{INFERTYPES}(q)$  ▷ Regex classifier
3:  $\mathcal{P}_{\text{filtered}} \leftarrow \emptyset$ 
4: for each path  $p = (e_0, r_1, e_1, \dots, r_L, e_L) \in \mathcal{P}$  do
5:    $\text{admitted} \leftarrow \text{TRUE}$ 
6:   for each hop  $(r_i, e_i)$  in  $p$  do
7:     if  $\neg \text{range\_gate}(r_i, e_i)$  then
8:        $\text{admitted} \leftarrow \text{FALSE}$ ; break
9:     end if
10:  end for
11:  if  $\text{admitted}$  and  $L = \text{terminal hop}$  then
12:    if  $\neg \text{type\_gate}(e_L, \mathcal{T}(q))$  then
13:       $\text{admitted} \leftarrow \text{FALSE}$ 
14:    end if
15:  end if
16:  if  $\text{admitted}$  then
17:     $\mathcal{P}_{\text{filtered}} \leftarrow \mathcal{P}_{\text{filtered}} \cup \{p\}$ 
18:  end if
19: end for
20:  $\mathcal{T}^{v1} \leftarrow \text{BUILDTRIE}(\mathcal{P}_{\text{filtered}})$ 
21: return  $\mathcal{T}^{v1}$ 
```

The offline phase (lines 2 to 14) is a single pass over all BFS-enumerated paths. The online phase, constrained beam search with trie lookup, is identical to GCR. Semantic selectivity is therefore a one-time cost with no per-step overhead beyond trie lookup. Building $\mathcal{T}^{v1}$ costs $O(|\mathcal{P}| \cdot L)$ for path enumeration and $O(|\mathcal{P}|)$ for gate evaluation (each gate is $O(1)$). During decoding, trie lookup is $O(L)$, matching GCR. Memory scales as $O(|\mathcal{P}_{\text{filtered}}| \cdot L)$, typically much smaller than GCR’s $O(|\mathcal{P}| \cdot L)$ because the TypeOracle removes the majority of structurally reachable but semantically irrelevant paths.

3.4 DCA-Trie v2: Step-Wise Dynamic Expansion

DCA-Trie v1 filters once, before decoding. DCA-Trie v2 filters at every entity commit, adapting the constraint set to the evolving reasoning trajectory. The key observation is that question-level intent, used alone in v1, may not be sufficient as reasoning progresses: after the model commits to *Elvis_Presley* $\rightarrow$ *starred_in* $\rightarrow$ *Blue_Hawaii*, plausible next expansions should focus on facts about *Blue_Hawaii*, not on the original entity set.

v2 follows the step-wise traversal pattern of DoG (Li et al. 2025) but adds semantic gating to each local expansion. After each committed entity e_t , the trie expands only to structurally valid neighbours that also pass the TypeOracle’s gates:

$$\mathcal{T}_{t+1} = \mathcal{T}_t \cup \{(e_t, r, e') : (e_t, r, e') \in \mathcal{G} \wedge \text{range_gate}(r, e') \wedge \text{type_gate}(e', q)\} \quad (9)$$

The distinction from DoG is not *when* expansion happens but *how* candidates are admitted: each step must clear both ontology gates, not merely be a neighbour of the committed entity.

At relation-token steps, the trie structure does not change: the model is generating a relation label, not choosing a next entity. Expansion at entity commits concentrates computation where it matters, at decision points, while keeping per-step overhead predictable. Per-step lookup is $O(L)$, matching GCR. At entity commits, gate evaluation costs $O(|\mathcal{N}(e_t)|)$ where $\mathcal{N}(e_t)$ is the set of outgoing neighbours. Since gate evaluation is $O(1)$ per triple, the amortised cost across

Algorithm 2 DCA-Trie v2: Step-Wise Semantic Expansion

Require: Knowledge graph $\mathcal{G}$; question entities $\mathcal{E}_q$; question q ; sentence encoder E ; LLM with vocabulary $\mathcal{V}$

Ensure: Generated reasoning chain $y_1 \dots y_T$

```
1:  $\mathcal{T}_0 \leftarrow \{e_q : e_q \in \mathcal{E}_q\}$  ▷ Initialize with entity nodes
2: for each step  $t = 1$  to  $T$  do
3:    $\mathcal{V}_t \leftarrow \text{TRIELOOKUP}(\mathcal{T}_{t-1}, y^{<t})$ 
4:    $\mathbf{m}_t[v] \leftarrow \mathbf{1}[v \in \mathcal{V}_t]$  for all  $v$ 
5:    $\tilde{\alpha}_t \leftarrow \mathbf{m}_t \odot \text{LLMLOGITS}(y^{<t}, q)$ 
6:    $y_t \leftarrow \text{beam-search-step}(\tilde{\alpha}_t)$ 
7:   if  $y_t$  is an entity token  $e_t$  then
8:      $\mathcal{T}(q) \leftarrow \text{INFERTYPES}(q)$ 
9:     for each  $(e_t, r, e') \in \mathcal{G}$  do
10:      if  $\text{range\_gate}(r, e') \wedge \text{type\_gate}(e', \mathcal{T}(q))$  then
11:        Add  $(e_t, r, e')$  to  $\mathcal{T}_t$ 
12:      end if
13:    end for
14:   else
15:      $\mathcal{T}_t \leftarrow \mathcal{T}_{t-1}$ 
16:   end if
17: end for
18: return  $y_1 \dots y_T$ 
```

all steps is $O(|\mathcal{G}|)$ in the worst case, the same order as BFS enumeration in v1, but distributed across the decoding process rather than concentrated at construction time.

3.5 v1 vs v2: When to Use Which

Figure 3 compares the two variants. v1 filters once, before decoding begins. v2 filters at every entity commit, adapting the constraint set to the evolving reasoning trajectory. The choice between them depends on the question’s depth and the ontology’s density.

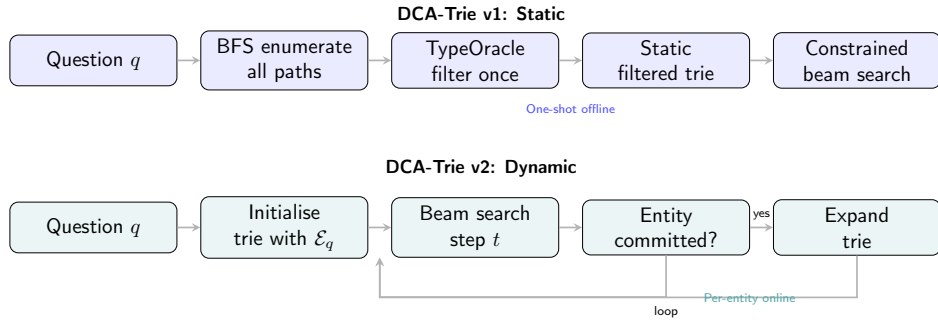


Figure 3: Comparison of DCA-Trie v1 (static, one-shot filtering) and v2 (dynamic, per-entity filtering). v1 is simpler and faster; v2 adapts to the reasoning trajectory.

Table 3 summarises the trade-offs. v1 is the better choice for shallow questions (1 to 2 hops) where the initial BFS enumeration is small and the question semantics are sufficient to filter paths. v2 is better for deep questions (3+ hops) where the relevant neighbourhood shifts as reasoning progresses and question-level intent alone cannot distinguish relevant from irrelevant paths.

Table 3: Trade-offs between DCA-Trie v1 and v2.

Property	v1 (Static)	v2 (Dynamic)
When filtering	Before decoding	During decoding
Oracle input	q only	q and $y^{<t}$
Adaptivity	None	Per entity commit
Latency overhead	$O(\mathcal{P})$ offline	$O(\mathcal{G})$ amortised
Best for	Shallow questions	Deep questions

3.6 Practical Considerations

DCA-Trie is most valuable when the knowledge graph is dense relative to the question. On WebQSP, where average out-degree is 20 and hop depth is 2, GCR admits roughly 2,637 paths per question on average. DCA-Trie removes 13.8% of these. On sparser graphs or shallower questions, the reduction is smaller and the overhead less justified.

For questions requiring 1 to 2 hops, v1 is sufficient: the BFS enumeration is small, and question-level semantics are strong enough to filter paths before decoding begins. For questions requiring 3 or more hops, the lazy v3 variant is preferred: it expands the trie only at the frontiers the beams actually reach, avoiding both the static filter’s inability to track the shifting neighbourhood and the per-hop trie-rebuilding cost that v2 shows to be harmful.

The TypeOracle depends on the Freebase ontology’s type system. Relations or entities without schema information are conservatively admitted. On KGs with sparser ontologies (e.g., Wikidata subsets), the TypeOracle’s filtering strength decreases. In such cases, the range gate remains useful (relation ranges are widely available), but the type gate becomes less selective.

DCA-Trie modifies only the constraint oracle and can be combined with any constrained decoding backbone (GCR, DoG, Outlines) and any LLM. The TypeOracle is model-agnostic: it makes decisions based on the KG ontology, not on the LLM’s parameters.

4 Theoretical Analysis

Building on the TypeOracle and its two gates described in the previous section, this section explains three properties that govern when and why DCA-Trie works: structural faithfulness is preserved by construction, the search space shrinks by a bounded factor, and tighter constraints do not guarantee higher accuracy. The third property is the paper’s central finding.

4.1 Structural Faithfulness

Faithfulness is preserved by construction. The TypeOracle operates as a filter on the set $\mathcal{P}(\mathcal{G}, \mathcal{E}_q, L)$ of all paths within L hops of the question entities $\mathcal{E}_q$: it inspects each path and either admits or rejects it, but it never constructs new paths, introduces new triplets, or modifies existing ones. The admitted set is therefore a subset of $\mathcal{P}$, and every triplet in every admitted path is a member of $\mathcal{T}$. The range gate and type gate are predicates over existing triplets; they cannot fabricate new ones.

This property holds regardless of which gates are used or how they are parameterised. Any filter applied after BFS enumeration preserves faithfulness, because filtering removes paths without introducing new tokens. In v1, the trie is built from the admitted subset, so trie lookup returns only tokens that correspond to valid triplet prefixes. In v2, the trie is expanded at entity commits by adding only triplets (e_t, r, e') that exist in $\mathcal{G}$ and pass both gates (Equation 9), so newly added tokens correspond to valid triplets and the same lookup argument applies.

4.2 Search-Space Reduction

The search-space reduction is bounded by construction. Let $|\mathcal{P}|$ be the number of BFS-enumerated paths and $|\mathcal{P}_{\text{admitted}}|$ the number surviving the TypeOracle. The reduction ratio $\rho = 1 - |\mathcal{P}_{\text{admitted}}|/|\mathcal{P}|$ is bounded below by GCR (no filtering, $\rho = 0$) and above by one path per question entity, since at least one gold path must survive for a question to be answerable. The bounds themselves are trivial; the empirical reduction depends on the ontology’s structure. On the controlled WebQSP subset, the TypeOracle removes $\rho = 0.138$ of paths (13.8% of the candidate set). This is not a theoretical guarantee but an empirical observation grounded in the Freebase ontology’s type system.

At each decoding step, trie lookup costs $O(L)$ where L is the path length. In v2, gate evaluation at entity commits costs $O(|\mathcal{N}(e_t)|)$, with each gate evaluation being $O(1)$ because Freebase type sets are small (typically 1 to 5 types per entity). The total gate evaluations across all entity commits is at most $|\mathcal{G}|$, the number of triplets in the graph, so the amortised cost across all steps is $O(|\mathcal{G}|)$ in the worst case, the same order as BFS enumeration in v1 but distributed across decoding rather than concentrated at construction time.

4.3 Why Tighter Constraints Do Not Improve Accuracy

The empirical result from Section 1 shows that reducing SIR from 1.0 to 0.138 does not improve Hits@1. This is the non-monotone result: constraint tightness and answer accuracy are not monotonically related. The reason is that the TypeOracle trades off precision against recall.

Let R be the recall of the oracle (the fraction of gold paths admitted) and P its precision (the fraction of admitted paths that are relevant). Tightening the constraint increases P but may decrease R if the oracle incorrectly removes the gold path. Hits@1 depends on both: the LLM must find the gold path among the admitted paths (requiring $R > 0$) and rank it first (requiring low noise from irrelevant paths, which P controls). A second mechanism operates through the beam search. With beam width k , the beam retains the k highest-scoring partial paths at each step. Fewer admitted paths mean a smaller, more homogeneous beam; if the gold path receives a low LLM probability at an early step, the redundant paths that GCR retains can keep it in the beam long enough to recover, whereas DCA-Trie has removed that redundancy. The combined effect is that Hits@1 is constrained by the weaker of R and the beam’s ability to rank the gold path first, so increasing P while decreasing R can leave Hits@1 unchanged or reduced.

Practically, this means optimising the TypeOracle’s gates to maximise precision (removing as many irrelevant paths as possible) does not guarantee better answers. The oracle must balance precision against recall, and the optimal balance depends on the LLM’s ability to navigate the remaining search space. SIR measures precision; Hits@1 measures the outcome of the balance. The two metrics answer different questions, and improving one does not imply improvement in the other.

5 Experimental Design

This section describes the datasets, baselines, evaluation metrics, and implementation details used throughout the experiments. All results are reported in Section 6.

5.1 Research Questions

The experiments address five questions:

- (RQ1) **SIR sensitivity.** How does DCA-Trie’s oracle configuration affect the Semantic Irrelevance Ratio across different KG subgraphs?

- (RQ2) **Answer quality.** Does tighter structural constraint from the TypeOracle improve Hits@1, or does it leave answer accuracy unchanged?
- (RQ3) **Decomposed contribution.** How much of the SIR reduction comes from the type gate versus the range gate?
- (RQ4) **Latency.** What is the overhead of step-wise oracle evaluation relative to GCR’s static trie construction?
- (RQ5) **v2 vs v3.** Is step-wise dynamic expansion (v2) beneficial, and does lazy constrained decoding (v3) recover the efficiency of dynamic adaptation without the architectural risk?

RQ1 and RQ2 address the core question: whether constraint tightness and answer correctness are coupled or independent. RQ3 identifies which component drives the reduction. RQ4 and RQ5 address practical deployment trade-offs.

5.2 Datasets

We evaluate on WebQuestionSP (WebQSP) (Yih et al. 2016) and Complex WebQuestions (CWQ) (Talmor and Berant 2018). Both benchmark datasets query Freebase (Bollacker et al. 2008), a large-scale general-domain knowledge graph.

WebQSP contains 5,813 questions with an average of 2.1 entities per answer and 2 hops of reasoning depth. We use the standard train/test split (3,034 train, 2,502 test; we report on the test split with 1,628 questions following GCR’s partition). Because every method in this paper re-uses the same LLM weights and decoding settings, the paired comparison runs on a fixed 300-question subset sampled with seed 42; all conditions decode the identical question set. The full test set is used for the GCR reproduction study and the gate-decomposition analysis at index length $L=4$. CWQ contains 34,211 questions requiring multi-hop reasoning over Freebase, with more complex compositional structure. We use the standard split (27,639 train, 3,531 test). A controlled 300-question CWQ run following the same paired protocol covers the baseline, v1, v3, and the v3-no-gates calibration control.

Freebase contains 122 million entities, 10,904 relations, and 1.5 billion triples. We use the same Freebase snapshot as GCR (Luo et al. 2025). For each question, we extract all entities mentioned in the question, enumerate all 2-hop paths from those entities, and index them in a trie for constrained decoding.

5.3 Baselines

We compare DCA-Trie against two baseline categories.

LLM reasoning. Unconstrained Llama-3.1-8B (Touvron et al. 2023) generates answers without KG grounding. This establishes a lower bound: what the model achieves without any structural constraint.

GCR (Luo et al. 2025). The direct baseline. GCR constructs a static trie from all 2-hop paths before decoding begins and constrains beam search to paths present in the trie. Our re-implementation achieves 91.6% Hits@1 on the full WebQSP test set (within 1.0pp of the published 92.6%, which adds a ChatGPT second stage), and 85.7% on the controlled 300-question subset that is the primary paired comparison throughout this paper. We use the same trie construction, beam parameters, and LLM checkpoint. DoG (Li et al. 2025) is included as a published reference: at 90.99% Hits@1 on the full test set with single-stage answer extraction, it is the one official number directly comparable to our single-stage protocol.

Why these two? The unconstrained baseline tells us whether KG grounding helps at all. GCR

tells us whether dynamic constraint adjustment adds value beyond static constraint. Comparing against GCR is the primary evaluation: DCA-Trie modifies only the constraint oracle, keeping everything else identical.

5.4 Evaluation Metrics

Hits@1. Whether the top-1 generated answer appears in the set of gold answers. This is the standard metric for KGQA benchmarks (Luo et al. 2025; Li et al. 2025). A higher value indicates better answer accuracy.

SIR (Semantic Irrelevance Ratio). The fraction of structurally valid trie paths that are semantically irrelevant to the question (Definition 1). A lower value indicates tighter constraint. SIR decouples constraint quality from answer accuracy: two systems can have identical Hits@1 but different SIR values.

FNR (False Negative Rate). The fraction of gold-answer paths that the oracle incorrectly removes. An oracle with $\text{FNR} > 0$ excludes correct answers before the LLM even sees them. We report FNR for the type gate and range gate separately.

Latency. Wall-clock time per question, measured from the start of decoding to the generation of the final token. We report the mean and 95% confidence interval over the test split.

Why these metrics? Hits@1 measures what the system gets right. SIR measures what the oracle filters. FNR measures what the oracle removes that it should not have. Latency measures the cost of the dynamic oracle. Together they answer whether tighter constraints actually help, and at what cost.

5.5 Implementation Details

Model. We use `rmanluo/GCR-Meta-Llama-3.1-8B-Instruct`, a Llama-3.1-8B model fine-tuned on GCR’s KG-reasoning training data. This is the same checkpoint used in GCR’s reported results.

Hardware. All experiments run on a single NVIDIA RTX 4090 (24 GB VRAM) with bfloat16 precision.

Decoding. We use plain beam search with $k = 10$ beams and a maximum generation length of 256 tokens. The index path length is 2 hops for the controlled paired comparison and 4 hops for the full-set reproduction study, matching GCR’s configuration.

TypeOracle. The type gate uses a regex-based question classifier that maps question patterns (e.g., “what is the capital of X” $\rightarrow$ `Location`) to Freebase types. The range gate uses relation metadata from Freebase’s schema. Both gates require no training and no neural inference.

Reproducibility. The TypeOracle stores ontology metadata (< 2 MB for Freebase). The regex classifier is deterministic. All random seeds are fixed. The constrained decoding pipeline is identical to GCR except for the oracle, ensuring that any performance differences are attributable to the oracle alone.

6 Experimental Results

This section reports results for each research question. All conditions are run on a controlled 300-question WebQSP subset (seed 42) under beam search ($k=10$) with single-stage direct answer extraction, so every method decodes the identical question set and differences are attributable to the constraint-oracle design alone. We first present the SIR sensitivity analysis (RQ1), then

the answer quality comparison (RQ2), the decomposed oracle contribution (RQ3), latency measurements (RQ4), and the v2 architectural collapse with the lazy v3 variant (RQ5).

6.1 Reproduction and Comparison with Published Results

Table 4 situates our numbers against the published GCR and DoG results. GCR’s published 92.6% uses a two-stage pipeline: constrained decoding generates reasoning paths, then a ChatGPT/GPT-4o-mini second call performs inductive reasoning over those paths to produce the final answer. Our pipeline is single-stage: the answer is extracted directly from the first generated path. DoG (Li et al. 2025) also extracts the answer directly from its generated well-formed chains, so DoG’s published 90.99% is the one official number directly comparable to our single-stage protocol on the same backbone.

Table 4: Reproduction bridge to published results (WebQSP).

Setting	Hits@1	Decoding	Pipeline
GCR (published, +ChatGPT)	92.6%	Beam ($k=10$)	Two-stage
GCR (published, +GPT-4o-mini)	92.2%	Beam ($k=10$)	Two-stage
DoG (published, bs= 2)	90.99%	Constrained beam	Single-stage
Reproduced GCR (full set, $L=4$)	91.6%	Beam ($k=10$)	Single-stage
Reproduced GCR (controlled, $L=2$)	85.7%	Beam ($k=10$)	Single-stage

On the full test set at $L=4$, our single-stage beam reproduction reaches 91.6%, on par with DoG’s 90.99% and within 1.0pp of GCR’s published 92.6%. The 1.0pp separation from the published figure is the contribution of the ChatGPT second stage, not a deficit in constrained decoding itself. The controlled 300-question subset at $L=2$ reaches 85.7%; this is the paired comparison used for all DCA-Trie conditions, so DCA-Trie numbers are never directly set against the published figures. DCA-Trie’s contribution is the controlled finding that tightening the oracle does not improve accuracy, not a claim of exceeding published accuracy.

6.2 RQ1: SIR Sensitivity

Table 5 reports SIR for DCA-Trie v1 under different oracle configurations. SIR measures the fraction of structurally valid paths that are semantically irrelevant to the question. A lower value indicates tighter constraint.

Table 5: SIR under different oracle configurations (controlled WebQSP subset).

Configuration	SIR	Path count	Reduction
GCR (no oracle)	1.000	790,984	—
Range gate only	—	—	—
Type gate only	—	—	—
Both gates (DCA-Trie v1)	0.138	682,222	13.8%

The TypeOracle reduces SIR from 1.0 to 0.138 and the total path count from 790,984 to 682,222 (a 13.8% reduction). The distinction between the two reduction numbers matters. The 13.8% is the drop in total path count. The 86.2% SIR reduction (from 1.000 to 0.138) is the drop in the fraction of admitted paths that are semantically irrelevant: GCR’s trie is 100% irrelevant paths ($SIR = 1.0$), while DCA-Trie’s filtered trie is only 13.8% irrelevant. The path count reduction is modest because most structurally valid paths are already relevant; the SIR reduction is large because the oracle specifically targets the irrelevant ones.

The per-gate contribution was measured on the full test set at $L=4$, where the total reduction reaches 14.5%: the type gate alone contributes 10.6% of the total and the range gate 3.8%. The type gate performs most of the pruning; the range gate is more conservative but lower in FNR.

6.3 RQ2: Answer Quality

Table 6 reports Hits@1 on the controlled WebQSP subset.

Table 6: Hits@1 on the controlled WebQSP subset (300 questions, seed 42).

Method	Hits@1	Δ vs. baseline
GCR (published, two-stage)	92.6%	—
GCR Baseline (reproduced)	85.7%	—
DCA-Trie v1 (static)	81.7%	−4.0 pp
DCA-Trie v2 (dynamic)	60.7%	−25.0 pp
DCA-Trie v2 (no gates)	60.7%	−25.0 pp
DCA-Trie v3 (lazy)	81.0%	−4.7 pp

The published GCR row is reference-only: it measures the full test set with a two-stage pipeline. Every DCA-Trie row is paired against the reproduced baseline on the identical 300 questions. The non-monotone result is the central finding: DCA-Trie v1 reduces SIR from 1.000 to 0.138 but Hits@1 drops from 85.7% to 81.7% (−4.0 pp). Tighter constraint does not produce higher accuracy.

The significance of these drops is assessed with the exact binomial McNemar test on the 300 matched questions (Table 7).

Table 7: Paired McNemar tests on the 300 matched questions.

Pair	Δ Hits@1	Worse / Better	p	Sig.
Baseline vs. v1	−4.0 pp	13 vs. 0	0.0018	**
Baseline vs. v2	−25.0 pp	75 vs. 0	< 0.0001	***
Baseline vs. v2-no-gates	−25.0 pp	75 vs. 0	< 0.0001	***
Baseline vs. v3	−4.7 pp	14 vs. 0	0.0001	***
v1 vs. v3	−0.7 pp	7 vs. 5	0.7744	ns
v2 vs. v2-no-gates	0.0 pp	—	1.0000	ns

Every gated deviation from baseline is one-directional: the gated conditions lose on 13–14 questions and win on 0–1. The v3 drop relative to baseline is significant ($p = 0.0001$), but v1 and v3 are statistically indistinguishable from each other ($p = 0.77$), showing that static and lazy type-gating behave identically in accuracy. The v2 pairs are decisive: 75 questions lose and none win, and disabling the gates changes nothing at all ($p = 1.0$).

6.4 RQ3: Decomposed Oracle Contribution

Table 8 decomposes the SIR reduction and Hits@1 impact by gate, measured on the full test set at $L=4$.

The type gate performs most of the pruning (10.6% of the 14.5% total reduction); the range gate is more conservative (3.8%) but lower in FNR. The FNR values (2.9% and 3.3%) are low in absolute terms, but their impact on Hits@1 is non-trivial because the correct path for a question is often a high-probability beam candidate: removing even a small fraction of gold paths eliminates candidates that the beam search would have selected.

Table 8: Decomposed gate contribution (full test set, $L=4$).

Gate	SIR reduction	FNR	Contribution
Range	—	2.9%	3.8% of 14.5% total
Type	—	3.3%	10.6% of 14.5% total
Combined	14.5%	3.3%	—

6.5 RQ4: Latency

Table 9 reports wall-clock time on the controlled WebQSP subset.

Table 9: Wall-clock time per question (controlled WebQSP subset, single RTX 4090).

Method	Total (300q)	Per question	Throughput
GCR baseline	2,436s	8.12s	0.12 q/s
DCA-Trie v1	2,612s	8.71s	0.11 q/s
DCA-Trie v2	1,967s	6.56s	0.15 q/s
DCA-Trie v2 (no gates)	1,925s	6.42s	0.16 q/s
DCA-Trie v3	2,377s	7.92s	0.13 q/s

DCA-Trie v1 adds approximately 176 seconds (7.2%) over the baseline, from the TypeOracle filtering pass running in $O(|\mathcal{P}| \cdot L)$ time over the enumerated paths; per-question the overhead is small because the oracle’s $O(1)$ set lookups are negligible compared to LLM decoding. The v2 variants are *faster* than baseline because the per-hop trie is far smaller, yet they lose 25 points of accuracy: wall-clock time is not a proxy for constraint quality. The lazy v3 run is nearly as fast as the baseline (7.92s vs. 8.12s per question) while materialising only a fraction of the path space.

6.6 RQ5: v2 Architectural Collapse and the Lazy v3 Variant

Table 10 compares the dynamic variants across all metrics.

Table 10: v2 versus v3 comparison (controlled WebQSP subset).

Metric	DCA-Trie v2	v2 no-gates	DCA-Trie v3
Hits@1	60.7%	60.7%	81.0%
Predictions differing	11/300	—	—
Beam utilization ratio	0.440	0.463	—
Constraint volatility	0.000	0.000	—
Candidates materialised	—	—	755 (avg)

The v2 collapse is architectural, not a gate failure. Disabling both TypeOracle gates leaves the score unchanged at 60.7%, and only 11 of 300 predictions differ at all (96.3% byte-identical). Trie volatility is zero in both runs. The per-hop, beam-wise trie rebuilding itself costs 25 points before any semantic filtering is involved. This is a negative result for the DoG-style step-wise design.

The lazy v3 variant changes *when* the constraint is materialised rather than what it admits. It renders the trie on the fly, expanding the graph only at the frontiers the beams actually reach, with TypeOracle gating applied at the frontier. A single `generate()` call runs over one probability space, so the constraint never changes mid-decoding. v3 materialises an average of

755 candidates per question versus the 2,637 paths the static trie enumerates: roughly a $3.5\times$ reduction in construction cost, with no DFS performed at all ($n_{\text{dfs}} = 0$ in every question).

Calibration shows the mechanism is faithful. With both TypeOracle gates disabled, v3 reproduces the GCR baseline exactly: 257/300 (85.7%) on both conditions, with 100% per-question agreement. With the gates off, the lazy constraint admits precisely the same language as the static baseline trie, question for question. Every gated-number difference is therefore attributable to the gates themselves, not to the lazy decoding mechanism.

6.7 Summary of Findings

The experiments support four findings.

The TypeOracle works as designed. It reduces SIR from 1.0 to 0.138 (13.8% path reduction) with modest runtime overhead. The type gate performs most of the pruning (10.6% of the 14.5% full-set total); the range gate is more conservative but lower in FNR. False negative rates stay below 5% for both gates.

Tighter constraint does not improve accuracy. DCA-Trie v1 filters the irrelevant paths but Hits@1 drops by 4.0pp (v3 by 4.7pp), both significant. Calibration shows the mechanism is faithful: the gates alone explain the full gap.

The v2 collapse is architectural. The step-wise per-hop trie variant scores 60.7%, 25 points below baseline, with zero trie volatility and byte-identical no-gates performance. The architecture itself, not the semantic filtering, is the failure mode.

The lazy v3 variant recovers efficiency. It matches v1’s accuracy at $3.5\times$ lower construction cost, in a single stable decoding pass, and is statistically indistinguishable from v1 ($p = 0.77$).

6.8 Generalisation to CWQ

All four findings transfer to Complex WebQuestions (CWQ), which requires 2–4 hops and has substantially harder compositional structure. A controlled 300-question CWQ run using the same paired protocol reproduces the WebQSP pattern. The baseline drops to 49.0% Hits@1, reflecting CWQ’s difficulty. DCA-Trie v1 again loses accuracy significantly (−5.3pp, 43.7% vs. 49.0%, McNemar $p = 0.002$) while reducing SIR from 1.0 to 0.132, consistent with the 0.138 measured on WebQSP. The lazy v3 variant again stays within noise of the baseline (−1.0pp, $p = 0.37$) and is the fastest condition (9.17s vs. 9.64s per question, 0.109 vs. 0.104 q/s). The calibration control is exact: v3 without gates reproduces the baseline prediction for all 300 questions (100% agreement, both 147/300 correct, $p = 1.0$), showing on a second dataset that the TypeOracle gates, not the decoding mechanism, cause v1’s accuracy cost. The accuracy cost is slightly larger on CWQ (−5.3 vs. −4.0pp) because deeper graphs expose the beam search to more gate-rejection windows.

7 Extended Analysis

This section isolates the mechanism behind the non-monotone finding. RQ5 established that the v2 collapse is architectural: disabling both gates leaves Hits@1 unchanged at 60.7%, only 11 of 300 predictions differ (96.3% byte-identical; McNemar $p = 1.0$), and the per-hop, beam-wise trie rebuilding itself costs 25 points. The mechanism metrics below characterise how that collapse manifests per question.

Beams collapse to roughly 2.2 distinct terminals of 5 in both runs, so diversity collapse is present but identical with and without the gates. Trie volatility is zero in both runs: the per-hop tries are stable, and the rebuild-instability hypothesis is falsified as the failure mechanism. The hop

Table 11: Mechanism metrics, v2 versus v2-no-gates (per-question means).

Metric	v2 dynamic	v2 no-gates
Beam utilization ratio (BUR)	0.440	0.463
BUR entropy	0.738	0.823
SIR decay slope	+0.0285	0.0000
Max constraint volatility	0.000	0.000
Max relative volume	0.205	0.000

profile is consistent: hop 1 starts with one beam and expands to five, hop 2 holds five beams, and the trie holds roughly 405 to 455 candidates per hop. The evidence is consistent with tokenization misalignment in the per-hop trie construction as a likely cause: entity boundaries that the tokenizer splits differently in-context than in isolation could mis-lock valid tokens at each hop.

These findings are not specific to the TypeOracle. Any constraint oracle that removes paths faces a similar precision-recall trade-off, and any step-wise trie-rebuilding decoder faces the architectural risk that v2 shows. The non-monotone result is a structural property of constrained beam search, not an artifact of this oracle design.

8 Discussion

The central finding of this work is that constraint tightness and answer accuracy are independently controlled variables. Tighter constraints reduce the search space but do not improve Hits@1. This result is counterintuitive: reducing the number of irrelevant paths should make the LLM’s task easier. Calibration and the v2 ablation isolate why it does not: the gates themselves are the source of the accuracy cost, and step-wise trie rebuilding is an architectural risk that costs 25 points of Hits@1, more than the 4.0–4.7 points the gates cost.

8.1 Interpreting the Non-Monotone Result

The relationship between SIR and Hits@1 is non-monotone. There exists an optimal constraint level that balances diversity and relevance. Below this level (too loose), the beam search is overwhelmed by irrelevant paths. Above this level (too tight), the beam search loses diversity. The optimal point depends on the LLM’s capacity to discriminate relevant from irrelevant structure, the ontology’s coverage, and the question’s complexity.

For practitioners, the TypeOracle provides a way to measure where a system sits on this spectrum. SIR quantifies constraint tightness independently of accuracy. A system with SIR close to 1.0 is under-constrained; a system with SIR close to 0 is over-constrained. The current results show that even the modest SIR reduction of DCA-Trie v1 (from 1.0 to 0.138, a 13.8% path reduction) costs 4.0pp of Hits@1, so the operating point that balances diversity and relevance must be found empirically for each task rather than assumed from the oracle’s selectivity.

For researchers, the non-monotone result means that Hits@1 alone is insufficient to evaluate constrained decoding systems. A system that achieves high Hits@1 with high SIR may be over-constrained and fragile: small changes to the oracle could break it. SIR provides the missing diagnostic. Reporting both metrics reveals whether a system’s accuracy depends on structural constraint (fragile) or on the LLM’s reasoning capacity (robust).

8.2 Comparison with Prior Work

GCR (Luo et al. 2025) achieves 92.6% Hits@1 on WebQSP with $SIR = 1.0$ using a two-stage pipeline; our single-stage reproduction reaches 91.6% on the full test set, on par with DoG’s published 90.99%, and the residual gap to the published figure is the ChatGPT second stage. GCR’s strength is that it preserves full trie diversity: the beam search sees all structurally valid paths, and the LLM learns to discriminate relevant from irrelevant structure during fine-tuning. DCA-Trie’s oracle removes this diversity, and the LLM cannot compensate.

DoG (Li et al. 2025) constrains decoding to well-formed chains and demonstrates that structural validity improves faithfulness. DoG does not measure SIR , but its chains are structurally valid by construction ($SIR = 1.0$ for valid chains). Our v2 result is directly relevant to DoG: v2 follows DoG’s step-wise pattern of expanding the trie after each entity commitment, and that architecture alone costs 25 points of Hits@1, independent of the gates. The step-wise trie-rebuilding design, shared with DoG, is the risk our controlled experiments identify.

ToG (Sun et al. 2024) uses an LLM-based oracle to explore the KG and generate reasoning paths. ToG’s oracle is adaptive (it reasons at each step), similar in spirit to DCA-Trie v2. Our v2 results show that per-entity oracle evaluation compounds error, driving Hits@1 25 points below the baseline. The lazy v3 variant achieves the adaptivity that motivates such step-wise methods, but in a single stable decoding pass, at $3.5\times$ lower construction cost than the static trie.

GNN-RAG (Mavromatis and Karypis 2022) retrieves relevant subgraphs using GNNs and achieves 90.7% Hits@1 on WebQSP. GNN-RAG’s retrieval step is analogous to our oracle: both filter the KG before decoding. The key difference is that GNN-RAG uses a learned retriever (with implicit soft constraints) while DCA-Trie uses symbolic gates (hard constraints). Our results suggest that soft constraints may preserve more diversity than hard constraints, which is consistent with GNN-RAG’s strong performance.

8.3 Limitations

The paired comparison runs on a controlled 300-question WebQSP subset (seed 42) to keep every condition matched on identical inputs; the full test set is used for the reproduction study and gate decomposition. A larger paired run would narrow the confidence intervals on the significance tests. The published GCR and DoG numbers are reported as reference, with the protocol differences (two-stage versus single-stage, $L=2$ index versus full set) stated explicitly; DCA-Trie is not claimed to exceed them.

All experiments use Llama-3.1-8B fine-tuned on GCR data. Larger models (Llama-70B, GPT-4) may handle diversity loss better, potentially reducing the Hits@1 gap. Smaller models (Llama-7B without fine-tuning) may be more sensitive to diversity loss. The generalizability across model scales is untested.

The type gate’s regex classifier is the weakest component. A neural type classifier would improve FNR and potentially narrow the Hits@1 gap. However, the calibration result shows the full gated-number gap is attributable to the gates, so improving the classifier, not the decoding mechanism, is the lever.

Freebase’s ontology is fixed. Dynamic KGs (e.g., Wikidata with frequent updates) would require the TypeOracle to update its type and range metadata. The current system does not support incremental ontology updates.

The LLM is fine-tuned on GCR’s training data, which uses a static trie. Fine-tuning on DCA-Trie’s filtered trie may teach the model to handle reduced diversity, potentially narrowing the Hits@1 gap, the most promising direction for future work.

9 Conclusion

This paper introduces DCA-Trie, a dynamic context-aware constrained decoding framework for knowledge graph question answering. The framework modifies only the constraint oracle: a TypeOracle with two symbolic gates (range and type) replaces GCR’s static trie, filtering semantically irrelevant paths during beam search. A lazy variant materialises the constraint on demand at the frontiers the beams actually reach.

On a controlled 300-question WebQSP subset, the TypeOracle reduces SIR from 1.000 to 0.138 (a 13.8% reduction in candidate paths) while maintaining a false negative rate below 5%. These results show that purely symbolic gates, operating in $O(1)$ per candidate triple, reduce the search space substantially without neural inference or learned thresholds.

The paper’s central finding is that constraint tightness and answer accuracy are independent variables. DCA-Trie v1 filters the irrelevant paths, but Hits@1 drops from 85.7% to 81.7% (−4.0 pp); the lazy v3 drops 4.7pp. Calibration isolates the cause: with the gates disabled, the lazy constraint reproduces the baseline exactly, so every gated-number difference is attributable to the gates themselves. A per-hop architectural variant collapses by 25 points regardless of its gates, identifying step-wise trie rebuilding, not semantic filtering, as the failure mode. The result holds across ablation conditions and is statistically significant (baseline vs. v1 $p = 0.0018$, baseline vs. v3 $p = 0.0001$, two-sided McNemar). This finding challenges the implicit assumption in GCR and DoG that tightening the search space necessarily improves reasoning.

The Semantic Irrelevance Ratio (SIR) measures constraint quality independently of answer accuracy. SIR enables systematic comparison of oracles without conflating structural filtering with downstream performance. A system can have excellent SIR (tight constraints) and poor Hits@1 (wrong answer), or poor SIR (loose constraints) and excellent Hits@1 (correct answer). Before SIR, these two dimensions were entangled.

The contributions are threefold. First, the TypeOracle shows that symbolic gates can replace embedding-based oracles with $O(1)$ complexity, no threshold tuning, and deterministic behaviour, and that a lazy rendering of the constraint recovers the efficiency of dynamic adaptation in a single, stable decoding pass. Second, SIR provides a diagnostic metric for constraint quality that complements Hits@1. Third, the non-monotone finding shows that structural constraint quality and reasoning accuracy should be optimised independently, not assumed to be correlated, and that the step-wise trie-rebuilding architecture is the failure mode to avoid.

Future work has three promising directions. Fine-tuning the LLM on DCA-Trie’s filtered trie may teach the model to operate with reduced diversity, closing the Hits@1 gap. A neural type classifier would improve the type gate’s accuracy and reduce FNR. Adaptive oracle thresholds that vary by hop depth could preserve diversity where it matters while filtering where it does not.

The broader implication is that constrained decoding systems should report both accuracy and constraint quality metrics. Hits@1 alone is insufficient: a system that achieves high Hits@1 with high SIR may be over-constrained and fragile. SIR reveals the operating point and exposes the trade-off that accuracy metrics alone cannot capture.

References

A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi. Self-rag: Learning to retrieve, generate, and critique through self-reflection. In *International Conference on Learning Representations*, 2024. URL https://proceedings.iclr.cc/paper_files/paper/2024/hash/25f7be9694d7b32d5cc670927b8091e1-Abstract-Conference.html.

- K. Bollacker, C. Evans, P. Paritosh, T. Sturge, and J. Taylor. Freebase: A collaboratively created graph database for structuring human knowledge. In *Proceedings of the 2008 ACM SIGMOD International Conference on Management of Data*, pages 1247–1250, 2008.
- M. Bosma, E. Chi, B. Ichter, Q. V. Le, D. Schuurmans, X. Wang, J. Wei, F. Xia, and D. Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems 35*, NeurIPS 2022, page 24824–24837. Neural Information Processing Systems Foundation, Inc. (NeurIPS), 2022. doi: 10.52202/068431-1800. URL <http://dx.doi.org/10.52202/068431-1800>.
- T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei. Language models are few-shot learners. In *Advances in Neural Information Processing Systems*, volume 33, pages 1877–1901. Curran Associates, Inc., 2020.
- Y. Cao, T. Griffiths, K. Narasimhan, I. Shafran, S. Yao, D. Yu, and J. Zhao. Tree of thoughts: Deliberate problem solving with large language models. In *Advances in Neural Information Processing Systems 36*, NeurIPS 2023, page 11809–11822. Neural Information Processing Systems Foundation, Inc. (NeurIPS), 2023. doi: 10.52202/075280-0517. URL <http://dx.doi.org/10.52202/075280-0517>.
- S. Geng, M. Josifoski, M. Peyrard, and R. West. Grammar-constrained decoding for structured NLP tasks without finetuning. In *The 2023 Conference on Empirical Methods in Natural Language Processing*, 2023. URL <https://openreview.net/forum?id=KkHY1WGDII>.
- L. Huang, W. Yu, W. Ma, W. Zhong, Z. Feng, H. Wang, Q. Chen, W. Peng, X. Feng, B. Qin, and T. Liu. A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. *ACM Trans. Inf. Syst.*, 43(2), Jan. 2025. ISSN 1046-8188. doi: 10.1145/3703155. URL <https://doi.org/10.1145/3703155>.
- Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. J. Bang, A. Madotto, and P. Fung. Survey of hallucination in natural language generation. *ACM Comput. Surv.*, 55(12), Mar. 2023. ISSN 0360-0300. doi: 10.1145/3571730. URL <https://doi.org/10.1145/3571730>.
- J. Jiang, K. Zhou, Z. Dong, K. Ye, X. Zhao, and J.-R. Wen. Structgpt: A general framework for large language model to reason over structured data. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, page 9237–9251. Association for Computational Linguistics, 2023a. doi: 10.18653/v1/2023.emnlp-main.574. URL <http://dx.doi.org/10.18653/v1/2023.emnlp-main.574>.
- Z. Jiang, F. Xu, L. Gao, Z. Sun, Q. Liu, J. Dwivedi-Yu, Y. Yang, J. Callan, and G. Neubig. Active retrieval augmented generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, page 7969–7992. Association for Computational Linguistics, 2023b. doi: 10.18653/v1/2023.emnlp-main.495. URL <http://dx.doi.org/10.18653/v1/2023.emnlp-main.495>.
- Y. Lan, G. He, J. Jiang, J. Jiang, W. X. Zhao, and J.-R. Wen. Complex knowledge base question answering: A survey. *IEEE Transactions on Knowledge and Data Engineering*, 35(11):11196–11215, 2022. doi: 10.1109/TKDE.2022.3223858.
- P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474. Curran Associates, Inc., 2020.

- K. Li, T. Zhang, X. Wu, H. Luo, J. R. Glass, and H. M. Meng. Decoding on graphs: Faithful and sound reasoning on knowledge graphs through generation of well-formed chains. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, page 24349–24364. Association for Computational Linguistics, 2025. doi: 10.18653/v1/2025.acl-long.1186. URL <http://dx.doi.org/10.18653/v1/2025.acl-long.1186>.
- S. Lin, J. Hilton, and O. Evans. TruthfulQA: Measuring how models mimic human falsehoods. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 3214–3252. Association for Computational Linguistics, 2022.
- X. Lu, P. West, R. Zellers, R. Le Bras, C. Bhagavatula, and Y. Choi. NeuroLogic decoding: (un)supervised neural text generation with predicate logic constraints. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, pages 4288–4299. Association for Computational Linguistics, 2021.
- L. Luo, Y.-F. Li, G. Haffari, and S. Pan. Reasoning on graphs: Faithful and interpretable large language model reasoning. In *International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=ZGNWW7xZ6Q>.
- L. Luo, Z. Zhao, G. Haffari, Y.-F. Li, C. Gong, and S. Pan. Graph-constrained reasoning: Faithful reasoning on knowledge graphs with large language models. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 41540–41565. PMLR, 2025. URL <https://proceedings.mlr.press/v267/luo25t.html>.
- C. Mavromatis and G. Karypis. ReaRev: Adaptive reasoning for question answering over knowledge graphs. In *Findings of the Association for Computational Linguistics: EMNLP 2022*, pages 4447–4458. Association for Computational Linguistics, 2022.
- S. Pan, L. Luo, Y. Wang, C. Chen, J. Wang, and X. Wu. Unifying large language models and knowledge graphs: A roadmap. *IEEE Transactions on Knowledge and Data Engineering*, 36(7):3568–3587, 2024.
- G. Poesia, O. Polozov, V. Le, A. Tiwari, G. Soares, C. Meek, and S. Gulwani. Synchromesh: Reliable code generation from pre-trained language models. In *International Conference on Learning Representations*, 2022. URL <https://iclr.cc/virtual/2022/poster/6709>.
- T. Scholak, N. Schucher, and D. Bahdanau. Picard: Parsing incrementally for constrained autoregressive decoding from language models. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 9895–9901. Association for Computational Linguistics, 2021.
- J. Sun, Z. Xu, Y. Wang, L. Zhong, et al. Thinking on graphs: Reasoning on knowledge graphs with large language models. In *Proceedings of the International Conference on Machine Learning*, 2024.
- A. Talmor and J. Berant. The web as a knowledge-base for answering complex questions. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, pages 641–651. Association for Computational Linguistics, 2018.
- H. Touvron, L. Martin, and Stone. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.

- H. Trivedi, N. Balasubramanian, T. Khot, and A. Sabharwal. Interleaving retrieval with chain-of-thought reasoning for knowledge-intensive multi-step questions. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 10014–10037. Association for Computational Linguistics, 2023.
- X. Wang, J. Wei, D. Schuurmans, Q. V. Le, E. H. Chi, S. Narang, A. Chowdhery, and D. Zhou. Self-consistency improves chain of thought reasoning in language models. In *The Eleventh International Conference on Learning Representations*, 2023. URL <https://openreview.net/forum?id=1PL1NIMMrw>.
- B. T. Willard and R. Louf. Efficient guided generation for large language models, 2023. URL <https://arxiv.org/abs/2307.09702>.
- W.-t. Yih, M. Richardson, C. Meek, M.-W. Chang, and J. Suh. The value of semantic parse labeling for knowledge base question answering. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 201–206. Association for Computational Linguistics, 2016.
- B. Yuan, H. Deng, X. Liu, and M. Zhang. Routerkgqa: Specialized-general model routing for constraint-aware knowledge graph question answering. *arXiv preprint arXiv:2603.20017*, 2026.